

CENG560 — Cloud Technology

Fall 2025 - Term Project

03.01.2026

Dockerizing an Application

Project Report and Specifications

Instructor: Asst. Prof. Dr. Metin BALCI

Report by

Mustafa SAĞLAM — 250802001

1. Introduction

This report explains the steps taken to create and containerize a simple web application using Docker. The web application is simple e-commerce app that has a NodeJS backend, a React frontend with Vite framework as build system and a simple PostgreSQL database for persistent storage. The application then is containerized using Docker.

2. Application Specification

The web application consists of 3 main parts;

- **PostgreSQL Database**

The database includes 2 tables, products and users. The products table has the name, rating, cost and resource paths to static product image files while the users table has email, name and password information. Since cloud security is not the scope of this report, the data is completely in the open.

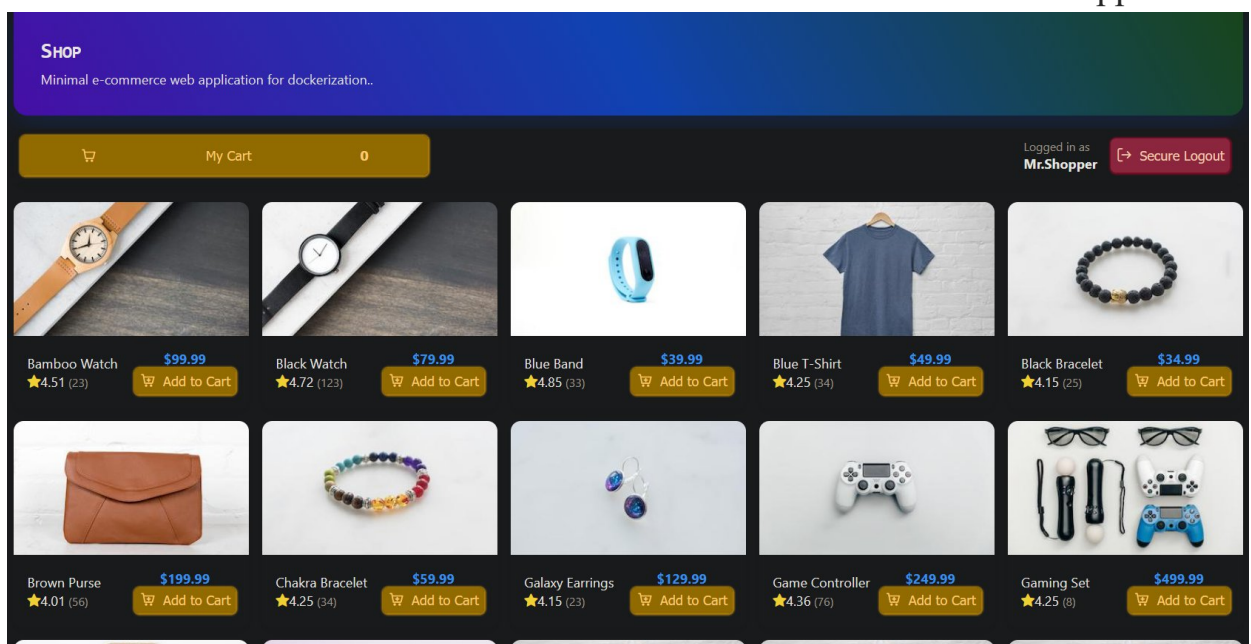
- **NodeJS Backend**

Project backend has an initial seed file that will seed the database tables explained above with initial data. The server file on the other hand includes API methods for logging in, signing up and getting a list of all the products.

- **ReactJS + Vite Frontend**

The frontend of the application is only one interactive page that lists the products along with their images, allowing for sign in and sign up operations. There is also a my cart feature allowing items to be added and checked out.

Below is a screenshot of the frontend of this e-commerce web application.



3. Dockerization and Resources Used

The dockerization of the application requires the creation of Dockerfiles and Docker Compose YAML file explained below. You can reach all the resources and project codes from below GitHub link.

<https://github.com/ixtiyer/CENG560-DockerSimpleWebApp>

Backend Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 4000
CMD ["npm", "start"]
```

Backend dockerfile uses node as base image and after installing the backend project dependencies listed in `package.json` exposes port 4000 to start an API server.

Frontend Dockerfile

```
# Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .

# Serve stage
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
# Optional: custom nginx config for SPA routing
COPY ./nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Frontend dockerfile on the other hand consists of two separate stages. In the build stage node is once again pulled as the base image and then the frontend project dependencies listed in `package.json` are installed. The second stage uses nginx as the base image and creates a routing table for the base localhost port 80 to start an Nginx application.

Docker Compose File

```
services:
  db:
    image: postgres:16-alpine
    container_name: shopdb
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: shopdb
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data

  backend:
    build: ./backend
    container_name: api
    environment:
      DATABASE_URL: postgres://postgres:postgres@db:5432/shopdb
      PORT: 4000
    depends_on:
      - db
    ports:
      - "4000:4000"

  frontend:
    build: ./frontend
    container_name: web
    depends_on:
      - backend
    ports:
      - "80:80"

volumes:
  db_data:
```

In this compose file there are 3 services and 1 volume is declared. The services are the postgres database service and its specifications, backend service that is tied to the database service and is baked using the previous dockerfile and frontend service that is tied to the backend and is baked using the previous dockerfile. The volume is created to make the postgres storage persistent.

The project can be run using the following command.

docker compose up -d

-d flag is used to detach the docker process from the terminal